LAPORAN PRAKTIKUM

STRUKTUR DATA

“ADT”

DISUSUN OLEH:

ABDUL JABBAR

2511533021

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan/tugas mengenai Abstract Data Type (ADT) dalam Bahasa Java dengan baik dan tepat waktu.

Laporan ini dibuat untuk memenuhi tugas mata kuliah Struktur Data yang membahas tentang konsep Abstract Data Type (ADT) serta implementasinya menggunakan bahasa pemrograman Java dengan memanfaatkan struktur data seperti Queue, Stack, ArrayList, dan LinkedList. Melalui tugas ini, saya dapat memahami bagaimana konsep ADT digunakan untuk merancang sebuah sistem yang terstruktur, modular, dan mudah dikembangkan dalam pemrograman.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi materi. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa yang akan datang.

Saya juga mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data serta semua pihak yang telah membantu dalam penyusunan laporan ini.

Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep Abstract Data Type (ADT) dalam Java**.**

Padang,1 april 2026

Abdul jabbar

DAFTAR ISI

KATA PENGANTAR ii
BAB I PENDAHULUAN 0
1.1 Latar Belakang 0
1.2 Tujuan 0
1.3 Manfaat 1
BAB II PEMBAHASAN 1
2.1 Pengertian ADT 1
2.2 Penjelasan kode program ADT 1
BAB III PENUTUP 6
3.1 Kesimpulan 6
3.2 Saran 7
DAFTAR PUSTAKA 7

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya sistem pemrograman yang terstruktur, efisien, dan mudah dipahami. Dalam dunia pemrograman, khususnya pada mata kuliah Struktur Data, diperlukan konsep yang dapat membantu dalam mengelola data secara sistematis dan terorganisir. Salah satu konsep penting yang digunakan adalah **Abstract Data Type (ADT)**. ADT merupakan konsep yang menggambarkan struktur data berdasarkan perilaku atau operasi yang dapat dilakukan terhadap data tersebut tanpa harus mengetahui bagaimana implementasinya secara detail.

Dalam bahasa pemrograman **Java**, konsep ADT sangat penting karena Java merupakan bahasa pemrograman berorientasi objek yang mendukung pembuatan kelas, metode, serta struktur data seperti **Queue, Stack, ArrayList, dan LinkedList**. Dengan menggunakan ADT, programmer dapat merancang sebuah program yang lebih modular, terstruktur, dan mudah dikembangkan. ADT memungkinkan pemisahan antara konsep data dan implementasi, sehingga program menjadi lebih fleksibel dan mudah dipelihara.

Penerapan ADT dalam Java sering digunakan dalam berbagai sistem, seperti sistem perpustakaan, sistem antrian, manajemen data mahasiswa, dan lain sebagainya. Melalui penggunaan struktur data seperti Queue untuk peminjaman, Stack untuk pengembalian, dan LinkedList atau ArrayList untuk penyimpanan data, program dapat berjalan lebih efisien dan sesuai dengan kebutuhan sistem.

Oleh karena itu, pemahaman mengenai ADT dalam Java sangat penting bagi mahasiswa Informatika, khususnya dalam mempelajari Struktur Data. Dengan memahami konsep ADT, mahasiswa dapat merancang program yang lebih terstruktur, efisien, dan sesuai dengan prinsip pemrograman yang baik. Latar belakang inilah yang menjadi dasar dalam pembuatan tugas mengenai **Abstract Data Type (ADT) dalam Java**.

1.2 Tujuan

1. Memahami konsep **Abstract Data Type (ADT)** dalam pemrograman Java.
2. Mengetahui fungsi dan peran ADT dalam pengelolaan struktur data.
3. Mempelajari penerapan ADT menggunakan bahasa pemrograman Java.
4. Mengimplementasikan struktur data seperti **Queue, Stack, ArrayList, dan LinkedList** dalam sebuah program.

5. Melatih kemampuan mahasiswa dalam merancang program yang terstruktur, modular, dan efisien.

1.3 Manfaat

Bagi Mahasiswa

Dapat menambah pemahaman mengenai konsep Abstract Data Type (ADT) serta penerapannya dalam bahasa pemrograman Java, khususnya dalam penggunaan struktur data seperti Queue, Stack, ArrayList, dan LinkedList.

Bagi Pembaca

Memberikan wawasan dan informasi mengenai pentingnya ADT dalam pemrograman Java serta bagaimana implementasinya dalam pembuatan program yang terstruktur dan efisien.

Bagi Penulis

Melatih kemampuan dalam menganalisis, merancang, dan mengimplementasikan konsep ADT ke dalam program Java serta meningkatkan keterampilan dalam penyusunan laporan ilmiah.

Bagi Dunia Pendidikan

Dapat menjadi referensi atau bahan pembelajaran dalam memahami konsep Struktur Data dan penerapan ADT dalam pemrograman Java.

BAB II PEMBAHASAN

2.1 Pengertian ADT

Tipe data abstrak (ADT) dapat didefinisikan sebagai *model matematika dari objek data yang menyempurnakan tipe data dengan cara mengaitkannya dengan fungsi-fungsi yang beroprasi pada data yang bersangkutan*. Merupakan hal yang sangat penting untuk mengenali bahwa operasi-operasi yang akan dimanipulasi data pada objek yang bersangkutan termuat dalam spesifikasi ADT. Sebagai contoh, ADT HIMPUNAN didefinisikan sebagai koleksi data yang diakses oleh operasi-operasi himpunan seperti penggabungan (UNION), irisan (INTERSECTION), dan selisih antar-himpunan (SET DIFFERENCE).

2.2 Penjelasan kode program ADT

Pada praktikum pekan 1 mahasiswa informatika kelas c di minta untuk membuat kode program seperti gambar yang di bawah ini

```java
package pekan1_2511533021;

public final class Jam_2511533021 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    // --- Validator ---
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);

    }

    // --- Konstruktor/Kreator ---
    public Jam_2511533021(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;

    }

    // --- Selektor ---
    public int getHour() { return hh; }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    // --- Mutator--
    public void setHour(int h) { this.hh = h; }
    public void setMinute(int m) { this.mm = m; }
    public void setSecond(int s) { this.ss = s; }

    // --- Konversi ---
    public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511533021 fromSeconds (int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %= 24 * 3600; // wrap 24 jam
        int h = total / 3600 ; total %= 3600;
        int m = total / 60; int s = total % 60;
```

Gambar 1.1

class Jam_2511533021 dengan modifier public final, yang berarti class tersebut dapat diakses dari luar package tetapi tidak dapat diwariskan atau di-extends oleh class lain. Class ini digunakan untuk merepresentasikan waktu yang terdiri dari jam, menit, dan detik.

Di dalam class terdapat tiga atribut yaitu hh, mm, dan ss yang bertipe integer. Variabel hh menyimpan nilai jam dengan batas 0 sampai 23, mm menyimpan nilai menit dengan batas 0 sampai 59, dan ss menyimpan nilai detik dengan batas 0 sampai 59. Ketiga variabel ini dibuat dengan modifier private agar tidak bisa diakses langsung dari luar class, sehingga data tetap aman dan hanya bisa diakses melalui method tertentu sesuai konsep enkapsulasi pada OOP.

Selanjutnya terdapat method isValid(int h, int m, int s) yang berfungsi sebagai validator untuk mengecek apakah nilai jam, menit, dan detik yang dimasukkan

sudah sesuai dengan aturan atau belum. Method ini bertipe boolean dan bersifat static, sehingga dapat dipanggil tanpa membuat objek terlebih dahulu. Jika nilai jam berada antara 0 sampai 23, menit antara 0 sampai 59, dan detik antara 0 sampai 59 maka method akan mengembalikan nilai true, sedangkan jika salah satu tidak sesuai maka akan mengembalikan false.

Setelah itu terdapat konstruktor Jam_2511533021(int h, int m, int s) yang digunakan untuk membuat objek jam. Konstruktor ini menerima tiga parameter yaitu h, m, dan s yang kemudian dimasukkan ke dalam atribut hh, mm, dan ss menggunakan keyword this. Fungsi konstruktor ini adalah untuk menginisialisasi nilai jam, menit, dan detik saat objek pertama kali dibuat.

```java
    // --- Konversi ---
    public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
    public static Jam_2511533021 fromSeconds (int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %= 24 * 3600; // wrap 24 jam
        int h = total / 3600 ; total %= 3600;
        int m = total / 60; int s = total % 60;
        return new Jam_2511533021(h, m, s);


    }

    // --- Relasional ---
    public int compareTo(Jam_2511533021 other) { return Integer.compare(this.toSeconds(), other.toSeconds()); }
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511533021 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;

    }
    public int hascode() { return java.util.Objects.hash(hh, mm, ss); }

    // --- Aritmatika ---
    public Jam_2511533021 plus(Jam_2511533021 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511533021 minus(Jam_2511533021 other) {return fromSeconds(Math.floorMod(this.toSeconds() - other.toSecond
    public Jam_2511533021 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
    public Jam_2511533021 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n)); }
    public Jam_2511533021 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600)); }
    public Jam_2511533021 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 *

    // --- Durasi (detik) -- bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAk)
    public static int durasiDetik(Jam_2511533021 jaw, Jam_2511533021 jakh) { return jakh.toSeconds() - jaw.toSeconds(); }

    // overriding ke string
    public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }

}
```

Gambar 1.2

```java
package pekan1_2511533021;

public class JamDriver_2511533021 {
        public static void main(String[] args) {
            Jam_2511533021 a = new Jam_2511533021(23, 59, 50);
            Jam_2511533021 b = new Jam_2511533021(0, 0, 15);

            System.out.println("a           = " + a);
            System.out.println("b           = " + b);
            System.out.println("a+b         = " + a.plus(b));
            System.out.println("next 20s    = " + a.nextNSeconds(20));
            System.out.println("durasi(a,b) = " + Jam_2511533021.durasiDetik(a, b));
            System.out.println("a.compareTo(b) = " + a.compareTo(b));
        }

}
```

Gambar 1.3

1. Inisialisasi Objek

Pada langkah awal, program membuat dua objek jam. Objek a diatur pada waktu 23:59:50 (hampir tengah malam), sedangkan objek b diatur pada waktu 00:00:15. Penentuan waktu ini sengaja dilakukan untuk menguji bagaimana sistem menangani perpindahan hari atau "overflow" waktu.

2. Penjumlahan Waktu (a.plus(b))

Ketika metode a.plus(b) dipanggil, program menjumlahkan total detik dari jam a dan jam b. Karena jam a berada di akhir hari, penambahan 15 detik dari jam b akan membuat total waktunya melewati batas 24 jam. Berkat operasi modulo (% 86400) yang ada pada metode fromSeconds, hasilnya tidak akan menjadi 24:00:05, melainkan "berputar" kembali ke awal hari menjadi 00:00:05.

3. Maju Beberapa Detik (a.nextNSeconds(20))

Langkah ini mirip dengan operasi penjumlahan. Program mencoba memajukan waktu pada objek a sebanyak 20 detik. Karena posisi awal adalah 23:59:50, maka setelah 10 detik waktu mencapai tengah malam (00:00:00), dan 10 detik sisanya membuat waktu berakhir di 00:00:10.

4. Perhitungan Durasi (durasiDetik)

Metode ini menghitung selisih antara jam akhir (b) dan jam awal (a). Dalam logika kodenya, perhitungannya adalah total_detik_b - total_detik_a. Karena secara numerik total detik pada jam b (15 detik) jauh lebih kecil daripada jam a (86.390 detik), hasil yang ditampilkan akan berupa angka negatif yang besar (yaitu -86.375). Hal ini menunjukkan durasi mundur dari jam a menuju jam b.

5. Perbandingan Relasional (a.compareTo(b))

Terakhir, metode compareTo membandingkan besaran total detik antara kedua objek. Karena jam a memiliki total detik yang lebih banyak dibandingkan jam b, metode ini akan mengembalikan nilai positif (biasanya angka 1). Ini menunjukkan

bahwa dalam sistem koordinat waktu 24 jam yang dimulai dari 00:00:00, objek a berada "lebih jauh" atau lebih besar nilainya daripada objek b.

```java
package pekan1_2511533021;
import java.util.Scanner;

public class JamDriver2_2511533021 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam ===");

        // 1. Input Jam Pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511533021 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511533021 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)        : " + j1.toString());
        System.out.println("Jam 2 (String)        : " + j2.toString());
        System.out.println("Jam 1 dalam detik     : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik     : " + j2.toSeconds());

        // 4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status                : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status                : Jam 1 lebih awal (sebelum) Jam 2");
        } else {
            System.out.println("Status                : Jam 1 dan Jam 2 sama persis");
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2)     : " + Jam_2511533021.durasiDetik(j1, j2) + " detik");

```

```java
        Jam_2511533021 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya: " + jNext);

        Jam_2511533021 jPrev = j1.prevSecond();
        System.out.println("Jam 1 Detik Sebelumnya: " + jPrev);

        // 6. Operasi Penjumlahan Jam
        Jam_2511533021 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2         : " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai.");

    }
    /**
     * Prosedur pembantu untuk melakukan input dan validasi secara berulang
     * sampai user memasukkan angka yang benar (0..23, 0..59).
     */
    private static Jam_2511533021 buatJamDariInput(Scanner sc) {
        int h, m, s;

        while (true) {
            System.out.print("Masukkan Jam (0-23)   : ");
            h = sc.nextInt();

            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt();

            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt();

            // Memanggil method static isValid dari kelas Jam
            if (Jam_2511533021.isValid(h, m, s)) {
                return new Jam_2511533021(h, m, s);
            } else {
                System.out.println("[Error] Input tidak valid! Silakan ulangi.\n");
```

Program dimulai dengan menampilkan judul aplikasi, lalu memasukkan dua buah jam (Jam 1 dan Jam 2). Proses input dilakukan melalui prosedur buatJamDariInput, yang akan terus meminta angka jam, menit, dan detik sampai pengguna memberikan nilai yang valid sesuai batasan (jam 0–23, menit 0–59, detik 0–59). Setelah kedua jam berhasil dibuat, program menampilkan hasil berupa representasi string dari masing-masing jam serta konversi keduanya ke dalam satuan detik. Selanjutnya, dilakukan perbandingan menggunakan compareTo untuk menentukan apakah Jam 1 lebih awal, lebih lambat, atau sama persis dengan Jam 2. Program kemudian menghitung durasi selisih waktu antara Jam 1 dan Jam 2 dalam detik. Setelah itu, dilakukan operasi tambahan: nextSecond untuk menampilkan Jam 1 satu detik setelahnya, prevSecond untuk menampilkan Jam 1 satu detik sebelumnya, serta plus untuk menjumlahkan Jam 1 dan Jam 2. Di akhir eksekusi, scanner ditutup dan program menampilkan pesan bahwa proses telah selesai.

BAB III PENUTUP

3.1 Kesimpulan

ADT adalah konsep penting dalam pemrograman berorientasi objek yang berfungsi untuk mendefinisikan tipe data beserta operasi-operasi yang dapat dilakukan

terhadapnya, tanpa harus memperhatikan detail implementasi internal. Dalam Java, ADT biasanya diwujudkan melalui **class** yang menyembunyikan detail data menggunakan **encapsulation**, serta menyediakan metode-metode untuk berinteraksi dengan data tersebut. Dengan ADT, programmer dapat membuat struktur data yang lebih terorganisir, reusable, dan mudah dipahami. Contoh nyata adalah kelas Jam_2511533021 yang kamu gunakan: ia mendefinisikan data jam, menit, detik, sekaligus menyediakan operasi seperti toSeconds(), compareTo(), nextSecond(), dan plus(). Hal ini menunjukkan bagaimana ADT memisahkan antara *apa yang bisa dilakukan* dengan *bagaimana cara melakukannya*.

3.2 Saran

1. Pahami Konsep Dasar – Fokus pada ide bahwa ADT adalah kontrak: ia mendefinisikan operasi, bukan detail implementasi.

2. Gunakan Encapsulation – Selalu sembunyikan atribut dengan private dan sediakan metode getter/setter atau operasi khusus agar data tetap konsisten.

3. Latih dengan Studi Kasus – Buat ADT sederhana seperti Jam, Tanggal, atau Matriks, lalu kembangkan dengan operasi tambahan (misalnya penjumlahan, perbandingan, validasi).

4.Perhatikan Validasi Input – Pastikan setiap data yang masuk ke ADT sesuai aturan, seperti jam 0–23 atau menit 0–59, agar objek selalu valid.

5. Kembangkan Modularitas – Pisahkan ADT dari *driver program* sehingga ADT dapat digunakan kembali di proyek lain tanpa perlu banyak modifikasi.

6. Eksperimen dengan Pola Desain – Setelah memahami dasar ADT, coba kombinasikan dengan konsep lain seperti inheritance atau interface untuk memperluas fleksibilitas**.**

DAFTAR PUSTAKA

Shoffin. 2010. *Artikel WordPress*. Diakses dari https://shoffin.wordpress.com/2010/08/25/14/ pada 1 April 2026.